Project Name: Agentic Development Platform (Agentic Builder)

Date: 2025-12-29

Tester: Muhd. Yuslan Abubakar

Component: JupyterHub Orchestrator & Flowise Spawner

1. Executive Summary

Objective: Validate that the MVP can successfully orchestrate separate Flowise containers for at least 5 concurrent users.

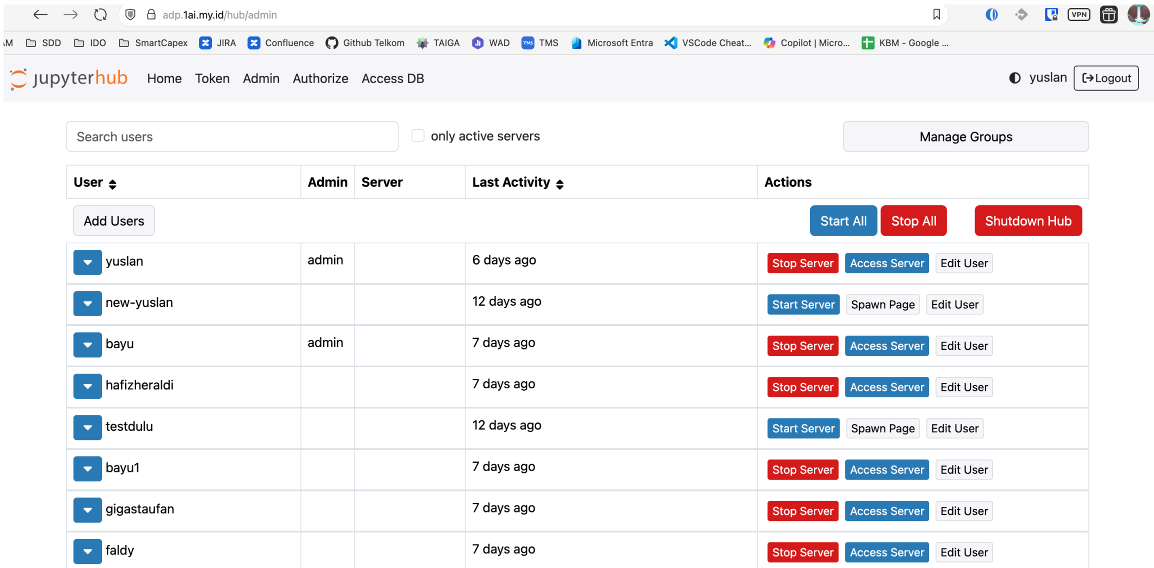
Result: PASS

Summary: The system successfully spawned 5 distinct Docker containers (one per user) upon login. All 5 environments were accessible simultaneously and remained stable.

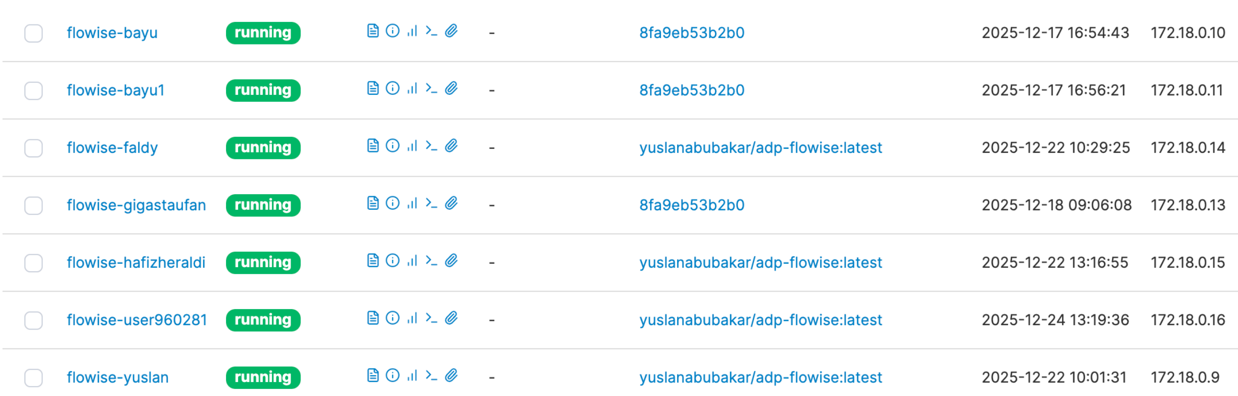
2. Architecture Validation (The Setup)

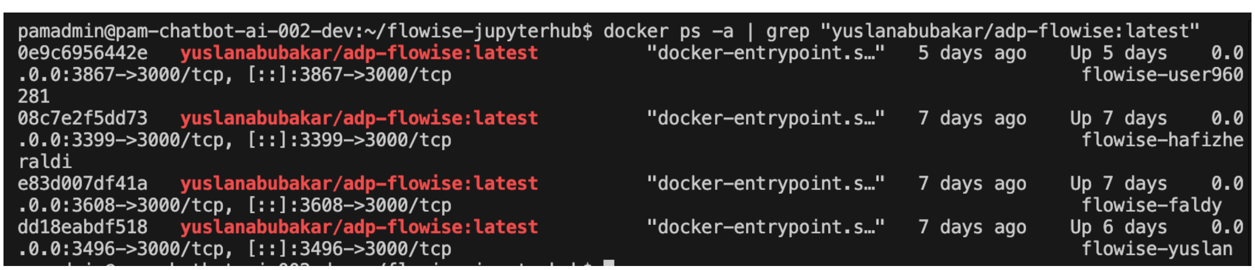
This section proves the architecture is deployed.

- **Orchestrator URL:** `https://adp.1ai.my.id` (JupyterHub)
- **Environment:** Docker on VM
- **Docker Network:** `jupyterhub-network`
- **Evidence:**



3. Concurrent Spawning Test (The Core Evidence)





Project Name: Agentic Development Platform (Agentic Builder)

Date: 2025-12-29

Tester: Muhd. Yuslan Abubakar

Component: JupyterHub Spawner & Resource Manager

1. Executive Summary

Objective: Validate that the system automatically provisions 1 unique PostgreSQL Schema and 1 dedicated S3 Bucket per user, ensuring 100% data isolation.

Result: PASS

Summary: The spawning mechanism successfully created distinct resources for User A (`user_alice`) and User B (`user_bob`). Access control tests confirmed that User A could not access User B's resources.

2. Architecture Logic (Visual Evidence)

This section explains "How" it works to the evaluator.

Mechanism: User Login → JupyterHub Spawner → Check/Create DB Schema → Check/Create S3 Bucket → Launch Flowise Container with specific Env Vars.

3. Test Case A: Database Schema Isolation

- **Test Scenario:** Two distinct users (`user_yuslan` and `user_bayu`) logged into the platform.
- **Expected Result:** PostgreSQL should contain two new schemas named `schema_yuslan` and `schema_bayu`.
- **Evidence:**

PostgreSQL@10.60.144.55 | flowise_shared | schema_yuslan | sql-1 (PostgreSQL@10.60.144...

- Info
- Schemas
- Event Triggers
- Extensions
- Storage
- System Info
- Roles

NAME	NAMESPACE ID	OWNER	COMMENT
public	2200	pg_database_owner	standard public schema
schema_bayu	23095	fw_user_bayu	
schema_bayu1	25170	fw_user_bayu1	
schema_faldy	26547	fw_user_faldy	
schema_gigastaufan	25853	fw_user_gigastaufan	
schema_hafizheraldi	23779	fw_user_hafizheraldi	
schema_new_yuslan	19942	fw_user_new_yuslan	
schema_newyuslan	19943	fw_user_newyuslan	
schema_slanbakar	19944	fw_user_slanbakar	
schema_testdulu	24462	fw_user_testdulu	
schema_user960281	28138	fw_user_user960281	
schema_yuslan	21722	fw_user_yuslan	

DELETE

4. Test Case B: Object Storage Bucket Isolation

- **Test Scenario:** The same two users uploaded a file via their Flowise interface.
- **Expected Result:** MinIO/S3 should show two distinct buckets `flowise-yuslan` and `flowise-bayu`.
- **Evidence:**

```
pamadmin@pam-chatbot-ai-002-dev:~/flowise-jupyterhub$ docke
c -c "
mc alias set myminio http://127.0.0.1:9000 minioadmin jVdLR
mc ls myminio/
"
Added `myminio` successfully.
[2025-12-16 03:16:37 UTC]     0B flowise-bayu/
[2025-12-17 09:56:21 UTC]     0B flowise-bayu1/
[2025-12-04 10:15:45 UTC]     0B flowise-faldy/
[2025-12-04 14:19:09 UTC]     0B flowise-fan/
[2025-12-18 02:06:08 UTC]     0B flowise-gigastaufan/
[2025-12-05 04:18:48 UTC]     0B flowise-hafiz/
[2025-12-16 03:29:57 UTC]     0B flowise-hafizheraldi/
[2025-12-04 01:33:14 UTC]     0B flowise-haha/
[2025-12-04 01:33:59 UTC]     0B flowise-hoho/
[2025-12-09 09:29:48 UTC]     0B flowise-new-yuslan/
[2025-12-09 09:53:43 UTC]     0B flowise-slan-bakar/
[2025-12-16 08:15:28 UTC]     0B flowise-testdulu/
[2025-12-24 06:19:36 UTC]     0B flowise-user960281/
[2025-12-04 00:16:17 UTC]     0B flowise-yuslan/
pamadmin@pam-chatbot-ai-002-dev:~/flowise-jupyterhub$
```

Project Name: Agentic Development Platform (Agentic Builder)

Date: 2025-12-29

Tester: Muhd. Yuslan Abubakar

Component: Custom MCP Server (Multi-DB Connector)

1. Executive Summary

Objective: Validate the deployment and functionality of the Custom MCP Server to execute Read/Write operations on PostgreSQL and MySQL.

Result: PASS

Summary: The Custom MCP Server successfully established connections to both databases. It performed a SELECT operation on PostgreSQL (Read) confirming full interoperability.

2. System Architecture (Evidence of Deployment)

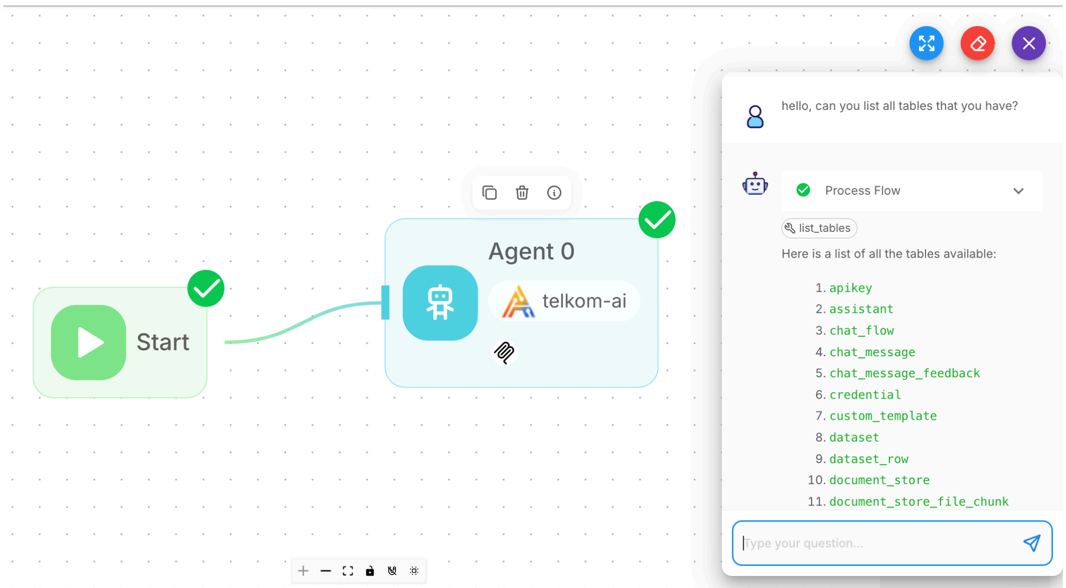
This section proves the MCP server is running and reachable.

- **MCP Server URL:** `http://mcp-server:9090/mcp`
- **Container Status:** `Up / Running`
- **Supported Tools List:**
  - `Describe_table`
  - `list_tables`

3. Test Case A: PostgreSQL Integration (List Tables)

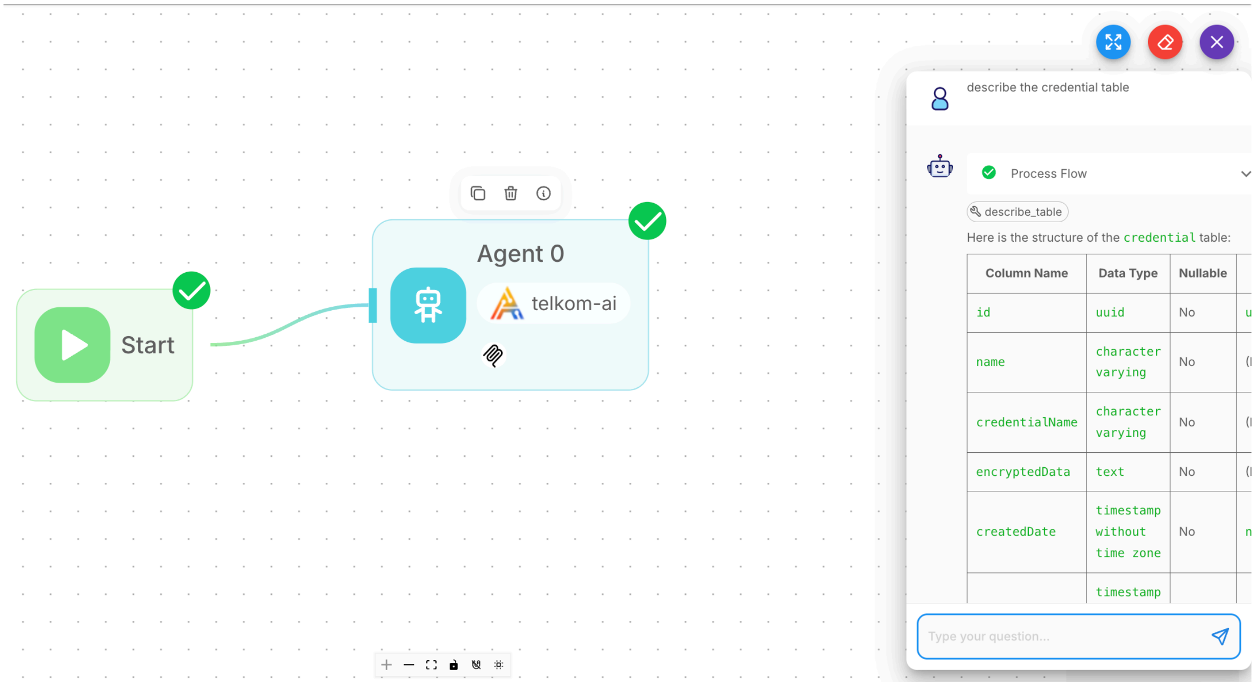
- **Database Type:** PostgreSQL
- **Target Schema:** `schema_yuslan`
- **Operation:** `List all tables`

- **Test Execution:**
    1. **Input:** User asks the chatbot: *"Hello, list all tables that you have"*
    2. **MCP Action:** The Agent calls `list_tables`.
    3. **Output:** The chatbot returns all tables on the schema.
- **Evidence:**



4. Test Case B: PostgreSQL Integration (Describe Tables)

- **Database Type:** PostgreSQL
- **Target Table:** `credential`
- **Operation:** `List all tables`
- **Test Execution:**
    1. **Input:** User asks the chatbot: *"Describe the credential table"*
    2. **MCP Action:** The Agent calls `list_tables`.
    3. **Output:** The chatbot returns the description of the credential table.
- **Evidence:**



Start
Agent 0
telkom-ai
describe the credential table
Process Flow
describe_table
Here is the structure of the credential table:
Column Name | Data Type | Nullable
id | uuid | No
name | character varying | No
credentialName | character varying | No
encryptedData | text | No
createdDate | timestamp without time zone | No
timestamp
Type your question...

Project Name: Agentic Development Platform (Agentic Builder)

Date: 2025-12-29

Tester: Muhd. Yuslan Abubakar

Component: ElasticSearch Vector Database Module

1. Executive Summary

Objective: Validate the successful integration of ElasticSearch as the central vector database for the Agentic Builder platform.

Result: PASS

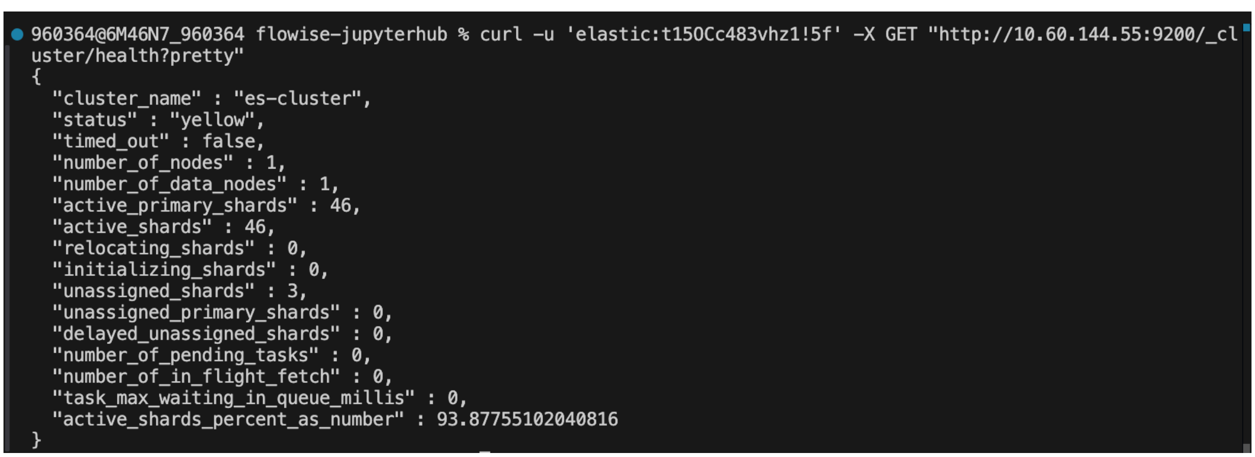
Summary: The system successfully connected to ElasticSearch, ingested 4 documents (PDF, TXT, CSV, DOCX), generated vector embeddings, and retrieved relevant context for user queries.

2. System Configuration (Evidence of Integration)

This section proves the connection is established.

Vector Index Name: `dokumen_okr`

Connection Status Screenshot:

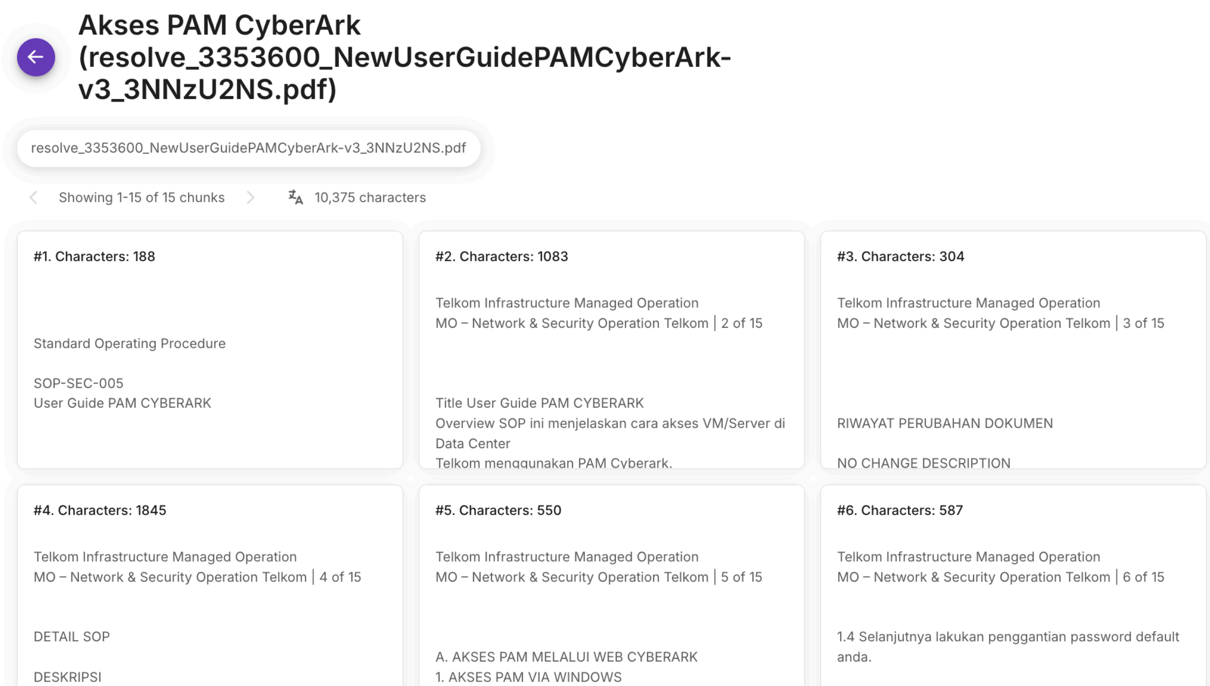


3. Data Ingestion Test (Evidence of Embedding)

This section proves the system accepts files and converts them into vectors.

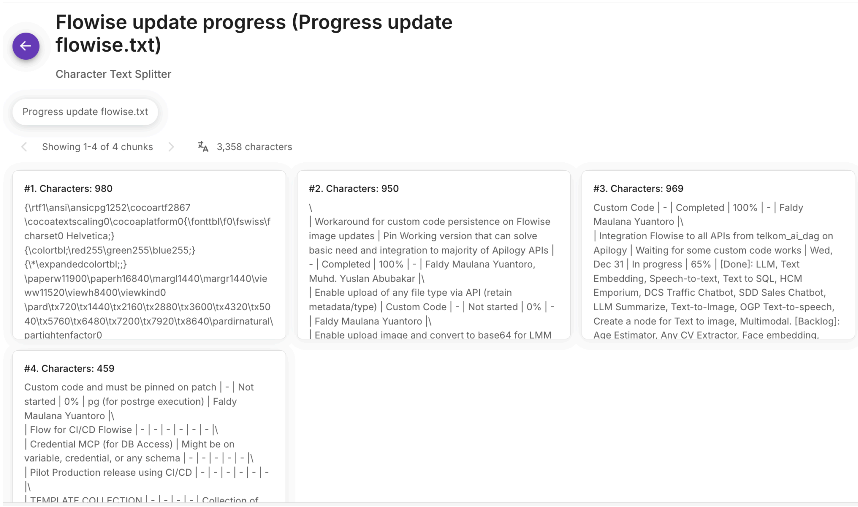
Test Case A: PDF Document

- **Filename:** NewUserGuidePAMCyberArk-v3_3NNzU2NS.pdf
- **File Size:** 1.4 MB
- **Upload Timestamp:** 2025-12-29
- **Evidence of Indexing:**



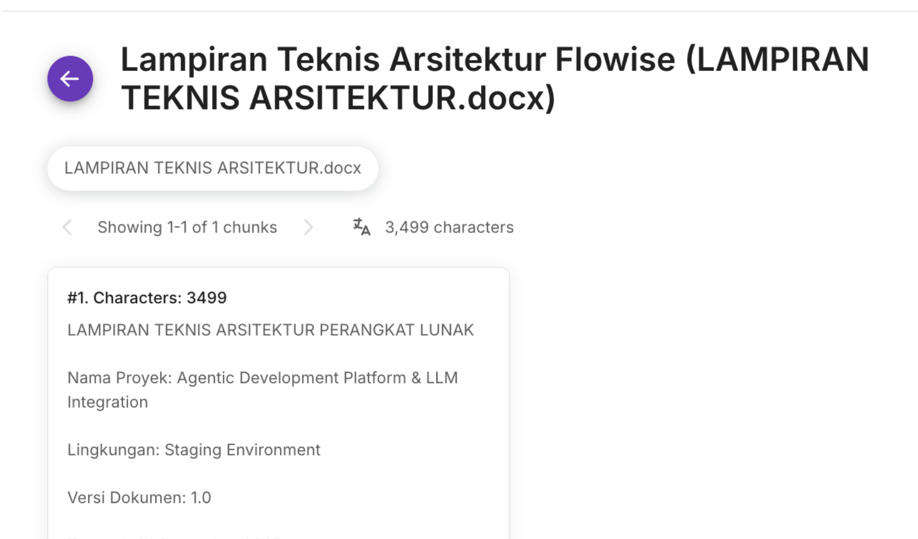
Test Case B: TXT Document

- **Filename:** `Progress update flowise.txt`
- **File Size:** 3 KB
- **Upload Timestamp:** 2025-12-29
- **Evidence of Indexing:**



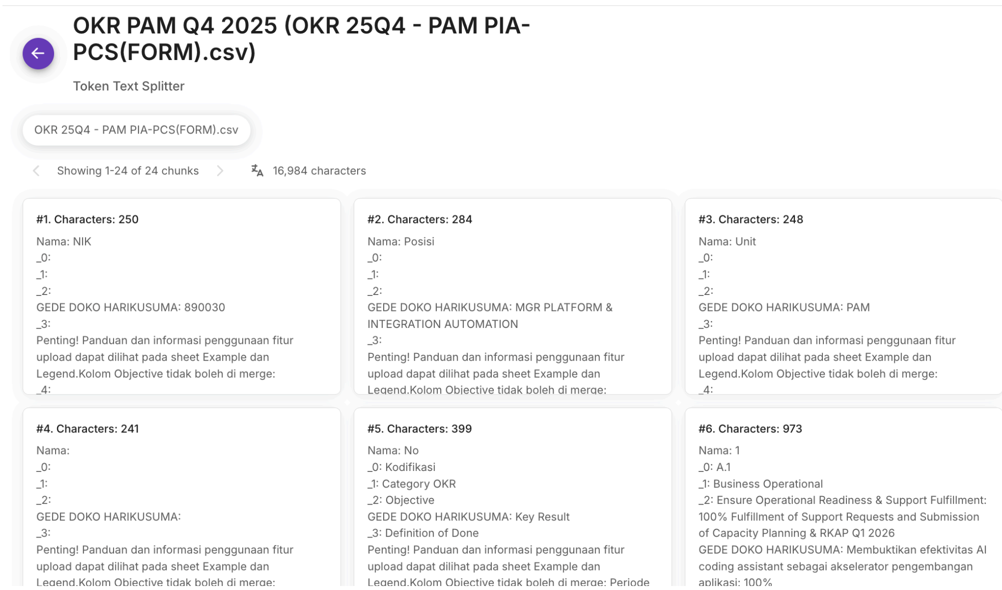
Test Case C: DOCX Document

- **Filename:** `Lampiran Teknis Arsitektur Flowise.docx`
- **File Size:** 119 KB
- **Upload Timestamp:** 2025-12-29
- **Evidence of Indexing:**



Test Case D: CSV Document

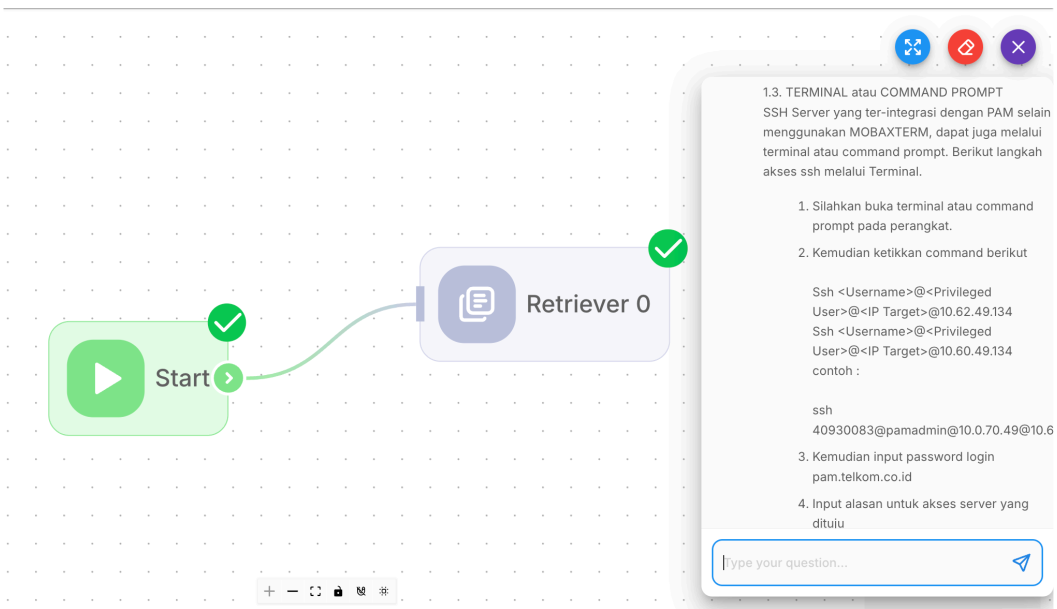
- **Filename:** `OKR PAM Q4 2025.csv`
- **File Size:** 12 KB
- **Upload Timestamp:** 2025-12-29
- **Evidence of Indexing:**



4. Retrieval & RAG Verification (Evidence of Retrieval)

This section proves the AI can "read" the vector database.

- **User Question:** "Akses PAM dengan terminal atau command prompt"
- **Expected Source:** NewUserGuidePAMCyberArk-v3_3NNzU2NS.pdf
- **System Response:**



LAMPIRAN TEKNIS ARSITEKTUR PERANGKAT LUNAK

Nama Proyek: Agentic Development Platform & LLM Integration

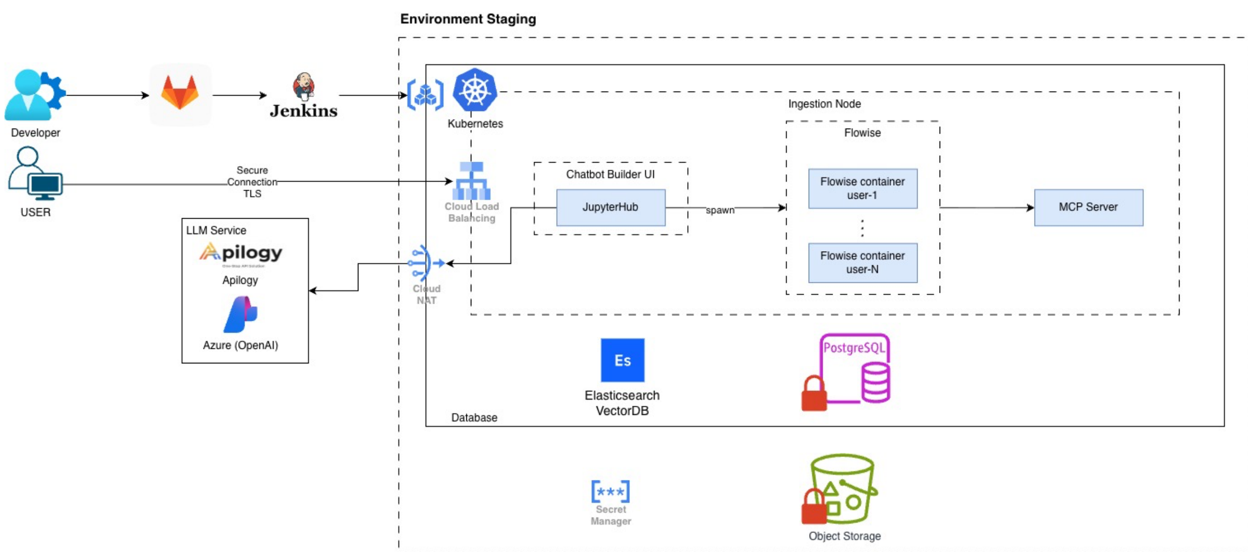
Lingkungan: Staging Environment

Versi Dokumen: 1.0

Tanggal: 23 Desember 2025

1. ARSITEKTUR SISTEM (SAD)

Bagian ini menggambarkan topologi arsitektur sistem pada lingkungan *Staging*. Sistem dirancang menggunakan prinsip *microservices* yang berjalan di atas platform Kubernetes.



2. DESKRIPSI KOMPONEN DAN LAYANAN

Sistem ini terdiri dari beberapa *layer* infrastruktur dan aplikasi. Berikut adalah rincian teknis setiap komponen:

2.1 DevOps & CI/CD Pipeline

Komponen untuk manajemen siklus pengembangan perangkat lunak.

Komponen	Fungsi Utama	Keterangan
GitLab	Source Code Management (SCM)	Repository utama untuk penyimpanan kode dan *version control*.
Jenkins	CI/CD Automation	Server otomatisasi yang melakukan *build image*, pengujian, dan *deployment* ke Kubernetes.

2.2 Infrastruktur Cloud & Network

Layanan dasar yang menangani orkestrasi, konektivitas, dan keamanan jaringan.

Komponen	Fungsi Utama	Keterangan
Kubernetes (K3s)	Container Orchestration	Menjalankan seluruh *workload* aplikasi (Flowise, JupyterHub, dll).
Cloud Load Balancing	Ingress Gateway	Pintu gerbang trafik masuk, menangani terminasi SSL/TLS.
Cloud NAT	Outbound Connectivity	Memfasilitasi akses internet aman untuk layanan internal (private IP) menuju API eksternal.
Secret Manager (vault)	Security	Penyimpanan terenkripsi untuk API Keys, kredensial database, dan sertifikat.

2.3 Komponen Aplikasi (Workload)

Inti logika bisnis dari platform *Chatbot Builder*.

Komponen	Fungsi Utama	Keterangan
JupyterHub	User Interface Spawner	Bertindak sebagai UI awal dan *spawner* yang memicu pembuatan container per pengguna.
Flowise	Chatbot App Builder	*Low-code tool* untuk membangun alur chatbot. Berjalan dalam mode *multi-tenant* (1 container per user).
MCP Server	RDBMS connector	*Model Context Protocol* server untuk menjembatani Flowise dengan RDBMS PostgreSQL dan MySQL eksternal.

2.4 Manajemen Data (Persistence)

Penyimpanan data terstruktur dan vektor.

Komponen	Fungsi Utama	Keterangan
Elasticsearch	Vector Database	Menyimpan *embedding* data untuk fitur *Retrieval-Augmented Generation* (RAG).
PostgreSQL	Relational Database	Menyimpan data user, sesi, konfigurasi flow, dan log transaksi.
Object Storage	File Storage	Penyimpanan untuk aset statis dan dokumen yang diunggah pengguna (S3 Compatible).

2.5 Layanan Eksternal (LLM Service)

Integrasi LLM Service Provider.

Komponen	**Fungsi Utama**	**Keterangan**
Apilogy	Aggregator API	Layanan solusi API terintegrasi.
Azure OpenAI	LLM Provider	Penyedia model LLM (GPT-3.5/GPT-4) kelas *enterprise*.

3. ALUR KERJA SISTEM

Mekanisme operasional sistem dari sisi *deployment* hingga interaksi pengguna:

1. **Deployment (CI/CD):** Kode yang diperbarui di *GitLab* akan diproses oleh *Jenkins* untuk kemudian diperbarui di *cluster Kubernetes*.
2. **Akses Pengguna:** Pengguna mengakses sistem melalui protokol HTTPS/TLS. Trafik didistribusikan oleh *Load Balancer* menuju *JupyterHub*.
3. **Inisialisasi Sesi:** *JupyterHub* membuat (*spawn*) kontainer *Flowise* yang terisolasi khusus untuk pengguna tersebut.
4. **Chatbot:**
    - Interaksi chat dikelola oleh *Flowise*.
    - Pencarian konteks data dilakukan ke *Elasticsearch*.
    - Inferensi AI dilakukan dengan memanggil *Azure OpenAI* atau *Apilogy*.
    - Koneksi dengan RDBMS eksternal (PostgreSQL dan MySQL) oleh Flowise melalui MCP Server.
5. **Storage:** Seluruh konfigurasi dan riwayat disimpan secara persisten di *PostgreSQL*.